

Flatcar Provisioning

Flatcar is configured at **first boot only** via declarative config. The primary toolchain is: **Butane YAML** → **Ignition JSON** → **applied by Ignition at boot**.

Toolchain Overview

```
Butane YAML (human-writable)
  ↓ butane / config-transpiler
Ignition JSON (machine-readable)
  ↓ passed as userdata / cloud metadata
Ignition service (runs in initramfs, first boot only)
  ↓
Configured Flatcar node
```

Ignition

Ignition runs in the **initramfs** on first boot only — before systemd starts. This means it can partition disks, write files, configure users, and set up systemd units before anything is running.

When Ignition runs:

- GRUB checks the EFI System Partition for `flatcar/first_boot`
- Sets `flatcar.first_boot=detected` kernel parameter
- Ignition runs, then deletes the `flatcar/first_boot` file
- Subsequent boots skip Ignition

Config sources (one at a time):

- Cloud provider instance userdata
- Network URL (`ignition.config.url` kernel parameter)
- QEMU fw_cfg (`opt/org.flatcar-linux/config`)
- VMware guestinfo (`ignition.config.data`)

Ignition vs cloud-init:

- JSON format (not YAML) — avoids type-coercion bugs
- Versioned config schema
- Runs once only — no variable substitution, no drift

Re-provisioning: Use `flatcar-reset` (available since Alpha 3535.0.0) to selectively clean rootfs state and re-trigger Ignition on next boot. Supports `--keep-paths` (regex) and `--keep-machine-id`.

```
sudo flatcar-reset --keep-machine-id --keep-paths '/etc/ssh/ssh_host_.*'
/var/log
sudo systemctl reboot
```

Butane / Container Linux Config (CLC)

Butane is the human-writable YAML layer that transpiles to Ignition JSON. The Flatcar variant (`variant: flatcar`) is the current standard; the older CLC format is also still supported.

Transpiling:

```
cat config.yaml | docker run --rm -i quay.io/coreos/butane:latest >
ignition.json
# or
butane < config.yaml > ignition.json
```

Common Butane sections:

Section	Purpose
<code>systemd.units</code>	Define/enable/disable systemd units
<code>storage.files</code>	Write arbitrary files
<code>storage.directories</code>	Create directories
<code>storage.links</code>	Create symlinks (used to disable sysexits)
<code>passwd.users</code>	Create users, add SSH keys
<code>passwd.groups</code>	Create groups
<code>kernel_arguments</code>	Set kernel cmdline args

Example — run NGINX container on boot:

```
variant: flatcar
version: 1.0.0
systemd:
  units:
    - name: nginx.service
      enabled: true
```

```

    contents: |
      [Unit]
      Description=NGINX example
      After=docker.service
      Requires=docker.service
      [Service]
      ExecStart=/usr/bin/docker run --name nginx1 --net host
docker.io/nginx:1
      Restart=always

```

Example — set update strategy:

```

variant: flatcar
version: 1.0.0
storage:
  files:
    - path: /etc/flatcar/update.conf
      overwrite: true
      mode: 0420
      contents:
        inline: |
          REBOOT_STRATEGY=etcd-lock

```

systemd-sysex

Sysex images extend the immutable base OS by mounting an overlay on `/usr`. This is the recommended way to add custom binaries to Flatcar without modifying the base image.

Official sysex extensions (in `/etc/flatcar/enabled-sysex.conf`):

Name	Available since	Default
containerd-flatcar	3794.0.0	Yes
docker-flatcar	3794.0.0	Yes
podman	3941.0.0	No
zfs	3913.0.0	No
python	4012.0.0	No
incus	4285.0.0	No
nvidia-drivers-*	4344.0.0	No

Disabling built-in Docker/containerd (e.g., for Kubernetes custom version):

```
variant: flatcar
version: 1.0.0
storage:
  links:
    - path: /etc/extensions/docker-flatcar.raw
      target: /dev/null
      overwrite: true
    - path: /etc/extensions/containerd-flatcar.raw
      target: /dev/null
      overwrite: true
```

Sysexth image format:

- Named NAME.raw (squashfs, ext4, or btrfs) or plain NAME/ directory
- Placed in /etc/extensions/ or /var/lib/extensions/
- Must contain usr/lib/extension-release.d/extension-release.NAME with ID=flatcar and SYSEXT_LEVEL=1.0 (for static binaries) or VERSION_ID=X.Y.Z (for dynamically linked)
- Binaries go in usr/bin/ , systemd units in usr/lib/systemd/system/

Community sysexth images: <https://github.com/flatcar/sysexth-bakery>

Runtime reload:

```
systemctl restart systemd-sysexth
```

Terraform

Flatcar supports Terraform-based provisioning. The provisioning/terraform/ docs cover provider-specific Terraform modules. Butane/Ignition configs are passed via Terraform's user_data field.

Boot Process Summary

```
GRUB
├─ checks EFI for flatcar/first_boot → sets kernel params
│   └─ initramfs
│       ├── randomizes disk GUID
│       ├── mounts root filesystem
│       └─ Ignition (if first_boot)
```

- └─ applies config, deletes first_boot flag
 - └─ pivot to userspace → systemd starts